

Manual de Usuario - Golampi Compiler

1. ¿Qué es este sistema?

Golampi Compiler es una aplicación web que permite escribir código Golampi, compilarlo, generar ensamblador ARM64, ejecutarlo con QEMU y visualizar errores, tabla de símbolos y salida real del programa.

2. Requisitos

- PHP 8 o superior.
- Navegador web moderno.
- Para ejecución ARM64 real, el proyecto ya incluye un toolchain local en Proyecto2/toolchain/.

3. Cómo iniciar el sistema

1. Abrir una terminal en la raíz del repositorio.
2. Ejecutar:

```
php -S 0.0.0.0:8000
```

3. Abrir en el navegador:
`http://localhost:8000/Proyecto2/frontend/index.html`

4. Cómo usar la GUI

Control	Uso
Nuevo	Limpia editor y paneles.

Cargar archivo	Permite seleccionar un archivo de código y cargarlo al editor.
Guardar código	Descarga el contenido actual del editor.
Compilar	Envía el código al backend y ejecuta el proceso completo.
Limpiar consola	Borra ensamblador, salida y estado.

5. Qué muestra la interfaz

- **Editor:** donde se escribe el programa Golampi.
- **ARM64 generado:** muestra el ensamblador producido por el compilador.
- **Ejecución ARM64 / QEMU:** muestra la salida real del binario ejecutado.
- **Estado del codegen:** informa si la compilación fue exitosa o si existen limitaciones.

6. Cómo compilar código

1. Escribir o cargar un programa en el editor.
2. Pulsar **Compilar**.
3. Revisar:

- el ensamblador ARM64 generado
- la ejecución real en QEMU
- el estado del compilador

7. Reportes descargables

- **Errores:** listado tabular de errores léxicos, sintácticos o semánticos.
- **Tabla de símbolos:** identificadores, tipo, ámbito, línea, columna y offset.
- **ARM64:** archivo .s generado por el compilador.
- **Salida:** resultado textual de la ejecución ARM64.

8. Ejemplo de uso

```
func main() {  
  
    fmt.Println(5)  
  
}
```

Resultado esperado en la consola de ejecución:

```
5
```

9. Capturas

Golampi Compiler

Editor

```
func addf(a float32, b float32) float32 {  
    return a + b  
}  
  
func main() {  
    x := 3.5  
    y := 2.5  
    fmt.Println(addf(x, y))  
    fmt.Println(true, "Hola", len("abc"), substr("Compilador", 0, 4), typeof(10), now())  
}
```

Editor

```
func addf(a float32, b float32) float32 {  
    return a + b  
}  
  
func main() {  
    x := 3.5  
    y := 2.5  
    fmt.Println(addf(x, y))  
    fmt.Println(true, "Hola", len("abc"), substr("Compilador", 0, 4), typeof(10), now())  
}
```



ARM64 generado

```
.section .rodata
bool_true: .asciz "true"
bool_false: .asciz "false"
type_int32: .asciz "int32"
type_bool: .asciz "bool"
type_string: .asciz "string"
type_float32: .asciz "float32"
type_rune: .asciz "rune"
type_nil: .asciz "nil"
fixed_now: .asciz "2026-04-14 00:00:00"
__newline: .ascii "\n"
__space: .ascii " "
__dot: .ascii "."
__minus: .ascii "-"
float_const_1000: .float 1000.0
float_lit_1: .float 3.5
float_lit_2: .float 2.5
str_lit_1: .asciz "Hola"
str_lit_2: .asciz "abc"
str_lit_3: .asciz "Compilador"

.section .bss
.align 3
print_buffer: .skip 128
substr_buffer: .skip 256
concat_buffer: .skip 512
int_buffer: .skip 32

.section .text
.align 2
# Runtime ARM64 para salida y cadenas
.global _start
_start:
# Entrada principal del compilador
bl main
# Finalizacion del proceso
mov x0, #0
mov x8, #93
svc #0

# Inicio de función addf
```

10. Observaciones

Si la compilación falla, la interfaz muestra una tabla con errores. Si la ejecución ARM64 falla, el mensaje aparece en el panel de ejecución sin romper el resto del pipeline.